

Vaja 4

Git in GitHub

Pripravi: Nejc Dolinar

1. Git

Kaj je Git

Git je porazdeljen sistem za upravljanje različic (distributed version control system) izvorne kode. Razvil ga je Linus Torvalds leta 2005 za potrebe razvoja Linux jedra. Git beleži zgodovino vseh sprememb v projektu (commits), omogoča delo več razvijalcem hkrati preko vej (branches), zlivanje sprememb (merge) in vrnitev v poljubno starejšo različico. Za razliko od centraliziranih sistemov ima vsak razvijalec na svojem računalniku celotno zgodovino projekta, kar omogoča delo brez stalne povezave s strežnikom.

Primeri uporabe

- Spremljanje sprememb v izvorni kodi skozi čas in zgodovina vseh različic
- Sodelovanje več razvijalcev na istem projektu brez prepisovanja sprememb
- Vejanje (branching) za razvoj nove funkcionalnosti brez vpliva na glavno vejo
- Vrnitev k prejšnji različici, če novi commit nekaj pokvari
- Postavljanje izdaj (releases) in označevanje različic z oznakami (tags)
- Integracija s CI/CD orodji – ob vsakem push-u se sprožijo testi in deploy
- Tudi za druge vrste besedilnih datotek: dokumentacijo, konfiguracije, šolske naloge

Osnovni Git ukazi za kreiranje novega repozitorija

```
git init          # inicializira nov Git repozitorij v trenutni mapi
git status        # prikaže stanje datotek (spremenjene, nove ...)
git add .         # doda vse spremembe v staging area
git add datoteka.txt # doda samo eno datoteko v staging
git commit -m "prvi commit" # potrdi spremembe v lokalno zgodovino
```

Hranjenje nove verzije (commit)

Po vsaki spremembi, ki jo želimo trajno shraniti, izvedemo commit:

```
git status        # preverimo, katere datoteke so spremenjene
git add .         # damo vse spremembe v staging
git commit -m "opis spremembe" # ustvarimo nov commit
```

Vsak commit dobi unikatni hash (npr. a3f5b1c...), avtorja, časovni žig in opisno sporočilo. Tako lahko kasneje natanko vidimo, kaj in kdaj se je zgodilo.

Nalaganje na GitHub (push)

```
git remote add origin https://github.com/uporabnik/repo.git
git branch -M main # preimenuje glavno vejo v main
git push -u origin main # pošlje commit-e na GitHub in nastavi sledenje
```

Po prvem push -u nastavitvi je za kasnejša nalaganja dovolj samo git push. Za avtentikacijo na GitHubu se uporablja personal access token (PAT) ali SSH ključ – uporabniško ime in geslo več ne delujeta.

Kloniranje GitHub repozitorija

Kloniranje pomeni, da iz GitHuba na svoj računalnik prenesemo celoten repozitorij skupaj z zgodovino:

```
git clone https://github.com/uporabnik/repo.git
git clone https://github.com/uporabnik/repo.git ime-mape
```

Z drugim načinom shranimo repozitorij v mapo z drugim imenom kot privzeto. Po kloniranju imamo lokalno kopijo, povezano z izvirnim remote (origin).

Prenos sprememb iz GitHub (pull)

Ko nekdo drug ali mi sami iz drugega računalnika potisnemo nove spremembe na GitHub, jih moramo na vsakem od svojih računalnikov potegniti k sebi:

```
git pull          # potegne najnovejše spremembe in jih zlije v lokalno vejo
git pull origin main  # eksplicitno določi remote in vejo
git fetch         # samo prenese info o spremembah, brez zlivanja
```

Pri pull lahko nastane konflikt, če smo lokalno spremenili iste vrstice kot nekdo na GitHubu – takrat ga moramo ročno rešiti.

2. GitHub

Kaj je GitHub

GitHub je spletna platforma za gostovanje Git repozitorijev. Omogoča oddaljeno shranjevanje kode, sodelovanje (pull requests, code review), upravljanje izdaj, sledenje napakam (issues), dokumentacijo (wiki) in CI/CD avtomatizacijo (GitHub Actions). Je najbolj razširjena Git platforma in v lasti Microsofta.

Kako kopiramo repozitorij na strežnik (GitHub) – postopek po korakih

1. Na GitHub.com kliknemo gumb »New repository« (zgoraj desno)
2. Določimo ime repozitorija, opis in vidnost (Public ali Private)
3. Izberemo, ali naj GitHub ustvari začetno README datoteko (priporočeno: ne, če imamo že lokalni projekt)
4. Kliknemo »Create repository« – GitHub pripravi prazen repozitorij in nam pokaže URL
5. Na lokalnem računalniku odpremo terminal v mapi s projektom
6. Inicializiramo Git: `git init` (če še ni)
7. Dodamo datoteke v staging: `git add .`
8. Ustvarimo prvi commit: `git commit -m "prvi commit"`
9. Povežemo lokalni repozitorij z GitHub remote: `git remote add origin <URL>`
10. Preimenujemo glavno vejo, če je potrebno: `git branch -M main`
11. Pošljemo vsebino: `git push -u origin main`
12. Po končanem push-u na GitHub strani osvežimo – vidimo datoteke in commit

Alternativa – GitHub Desktop

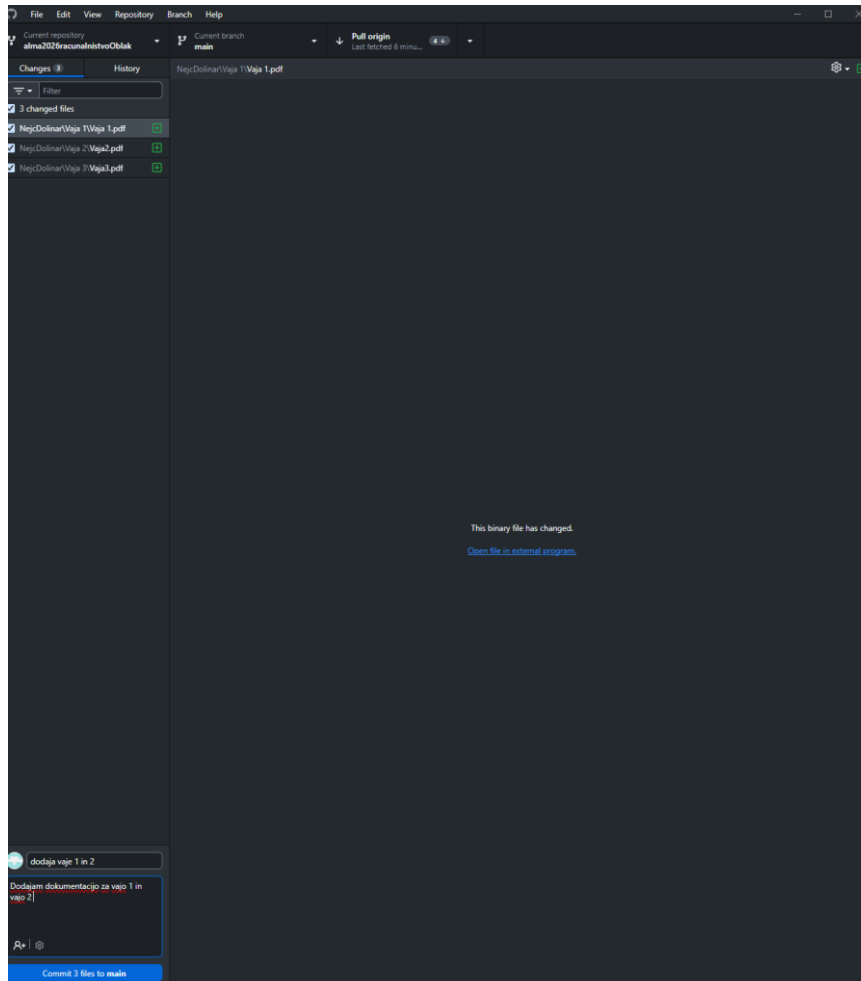
GitHub Desktop je grafični odjemalec, ki vodi skozi enake korake brez tipkanja ukazov:

- Repozitorij lahko ustvarimo direktno z »File → New repository« ali pa odpremo obstoječo mapo z »File → Add local repository«
- Spremenjene datoteke se prikažejo na levi (Changes); pred commit-om jih označimo s checkboxom
- Spodaj levo vpišemo kratko sporočilo in opsijski daljši opis, nato »Commit to main«
- Z gumbom »Publish repository« ali »Push origin« potisnemo spremembe na GitHub

- Z »Fetch origin« preverimo spremembe, z »Pull« jih potegnemo k sebi

Dokazi – izvedba na lastnem projektu

Za seminarsko nalogo sem si ustvaril repozitorij `alma2026racunalnistvoOblak` in vanj objavil tri vaje. Postopek sem izvedel v GitHub Desktopu.



GitHub Desktop pred commit-om: izbrane 3 datoteke (Vaja 1.pdf, Vaja2.pdf, Vaja3.pdf), commit z opisom »dodaja vaje 1 in 2« – pripravljeno za »Commit 3 files to main«

Vaja 1	dodaja vaje 1 in 2	now
Vaja 2	dodaja vaje 1 in 2	now
Vaja 3	dodaja vaje 1 in 2	now

GitHub stran repozitorija po push-u: vidijo se mape Vaja 1, Vaja 2 in Vaja 3 z istim commit sporočilom »dodaja vaje 1 in 2«

Povzetek

V Vaji 4 sem se naučil osnov Git-a (init, add, commit, push, pull, clone) in upravljanja repozitorijev na GitHubu. Za delo sem uporabil GitHub Desktop, ki grafično pokriva iste operacije kot Git CLI – ustvaril sem nov repozitorij, dodal datoteke vaj, naredil commit z opisom in jih poslal na GitHub. Sistem omogoča sledenje zgodovini sprememb in dostop do moje dokumentacije iz katerekoli naprave.